

Fundamentos de sistemas embebidos. Gpo. 2 Sem. 2026-2
317147531 – Sanches Diaz Daniel
Documentación Proyecto Final Consola Retro SDK
Enfoque en la Emulación

1. Objetivo

Diseñar e implementar una consola de videojuegos retro embebida en una Raspberry Pi 4, capaz de emular sistemas NES, SNES y Game Boy Advance mediante algún emulador, operando sobre Raspberry Pi OS Lite sin entorno de escritorio. El sistema arranca de forma autónoma al recibir alimentación, presenta al usuario una galería de juegos navegable exclusivamente mediante control, permite la carga de nuevas ROMs a través de memoria USB de forma automática, y se apaga de manera segura desde el propio menú.

2. Lista de materiales

Componente	Notas
Raspberry Pi 4 Model B (2 GB o más)	Procesador ARM Cortex-A72 64-bit
Tarjeta microSD (16 GB+, clase 10)	Sistema operativo y ROMs
Monitor o TV con entrada HDMI	Resolución mínima 720p
Cable micro-HDMI a HDMI	Incluido en kits oficiales
Fuente de alimentación 5 V / 3 A USB-C	Se recomienda fuente oficial
Gamepad USB compatible con SDL2	Ej. Xbox One, genérico USB HID
ROMs de NES, SNES y GBA (libres de derechos)	Homebrew o dominio público

Tabla 1. Hardware requerido

En cuanto al software se requiere:

- Raspberry Pi OS Lite (64-bit), sin entorno de escritorio
- Python 3.11 o superior
- pygame 2.x (python3-pygame)
- mednafen — emulador multi-sistema (apt install mednafen)
- SDL2 con driver kmsdrm (libsdl2-2.0-0, libsdl2-dev)
- ROMs libres de derechos para NES, SNES y Game Boy Advance (mínimo 15)

3. Antecedentes teóricos

3.1 Entrada de usuario en sistemas embebidos

En un sistema embebido de entretenimiento la entrada de usuario es el único mecanismo de comunicación entre el operador y el sistema. A diferencia de una computadora de propósito general, no existe teclado ni ratón: la interacción ocurre exclusivamente a través de un gamepad o joystick. Esta restricción impone que el software maneje correctamente eventos de dispositivos HID (Human Interface Device) sin depender de la presencia de un dispositivo específico [1].

El estándar HID define un protocolo USB genérico para dispositivos de entrada. Los gamepads compatibles con HID exponen sus controles como botones (valores binarios), ejes analógicos (valores en coma flotante entre -1.0 y $+1.0$) y hatswitch o D-pad (valores discretos de dirección). SDL2, y por extensión pygame, abstrae estos dispositivos bajo una API uniforme independientemente del fabricante [2].

3.2 Modelo de eventos de pygame

pygame implementa un modelo de programación orientado a eventos. Todos los eventos de entrada (teclado, ratón, joystick, ventana) se insertan en una cola interna (event queue) gestionada por SDL2. El programador debe leer esta cola regularmente para procesar los eventos pendientes. Si la cola no se vacía, SDL2 puede dejar de actualizar el estado interno de los dispositivos.

Existen dos formas de leer la cola en pygame:

- `pygame.event.get()`: extrae y devuelve todos los eventos pendientes en la cola, vaciándola.
- `pygame.event.pump()`: procesa la cola internamente sin devolver los eventos al programador. Es útil en bucles donde no se necesita procesar eventos pero sí mantener la cola activa para evitar que el sistema operativo marque la ventana como 'no responde'.

Los eventos relevantes para este subsistema son: KEYDOWN (tecla presionada), JOYBUTTONDOWN (botón de gamepad presionado), JOYHATMOTION (movimiento del D-pad), JOYAXISMOTION (movimiento de eje analógico), JOYDEVICEADDED y JOYDEVICEREMOVED (conexión/desconexión de gamepad en caliente).

3.3 Ejes analógicos y zona muerta

Los ejes analógicos de un gamepad rara vez devuelven exactamente 0.0 cuando el stick está en reposo debido a imprecisiones mecánicas y electrónicas del sensor (drift). Si se detectara cualquier valor distinto de cero como movimiento, el menú navegaría de forma errática sin que el usuario toque el control. Para resolver esto se define una zona muerta (dead zone): un umbral mínimo por debajo del cual el valor del eje se considera cero. En este sistema el umbral es 0.5, lo que corresponde al 50 % del recorrido máximo del eje [2].

3.4 mednafen — emulador multi-sistema

mednafen (My Emulator Doesn't Need A Frickin' Excellent Name) es un emulador de línea de comandos que soporta múltiples sistemas de videojuegos retro, incluyendo NES, SNES y Game Boy Advance. Se lanza con el comando:

```
mednafen -force_module <sistema> <ruta_rom>
```

El flag `-force_module` indica explícitamente qué núcleo de emulación utilizar, evitando que mednafen intente detectar automáticamente el sistema a partir de la extensión del archivo, lo que puede producir resultados incorrectos. mednafen usa SDL2 para renderizar su salida de video y capturar entrada, lo que lo hace compatible con el entorno de la Raspberry Pi sin escritorio [3].

3.5 Gestión del framebuffer entre procesos

En Raspberry Pi OS Lite el acceso al display se realiza mediante KMS/DRM (Kernel Mode Setting / Direct Rendering Manager). KMS garantiza acceso exclusivo al framebuffer: solo un proceso puede tener el control del display en un momento dado [4].

Cuando pygame está activo tiene el framebuffer. Al lanzar mednafen, si pygame no libera el framebuffer antes, mednafen no puede inicializar su propio display y falla con el error 'No available video device'. La solución es:

1. Llamar a `pygame.mixer.stop()` para detener el audio de pygame.
2. Llamar a `pygame.display.quit()` para liberar el framebuffer.
3. Esperar 1 segundo para que el kernel complete la liberación.
4. Lanzar mednafen con `subprocess.Popen()` y esperar con `.wait()`.
5. Al terminar mednafen, esperar 0.5 segundos adicionales.
6. Reinicializar pygame con `pygame.display.init()` y `pygame.mixer.init()`.
7. Recrear la superficie de pantalla completa con `display.set_mode()`.

3.6 subprocess.Popen vs subprocess.run

Python ofrece varias formas de lanzar subprocessos. `subprocess.run()` lanza el proceso y bloquea hasta que termina, devolviendo un objeto `CompletedProcess`. `subprocess.Popen()` lanza el proceso y devuelve inmediatamente un objeto `Popen` que representa el proceso en ejecución, permitiendo interactuar con él mientras corre (enviar señales, leer su salida, terminar). En este sistema se usa `Popen` porque el objeto proceso se almacena en `self.proceso`, lo que permite que el módulo `MonitorUSB` lo termine (`proceso.terminate()`) cuando se inserta un USB con ROMs mientras se está jugando [5].

4. Advertencias y cuidado de componentes

4.1 Seguridad eléctrica

- La Raspberry Pi 4 opera a 5 VCC. Por lo mismo no se recomienda que pases de ese voltaje, siempre utiliza el cargador que viene con la Raspberry.
- Al conectar el mando por USB se recomienda tener mucho cuidado para no dañar los puertos de la Raspberry.

4.2 Cuidado de la tarjeta microSD

- Siempre apague el sistema a través del menú de confirmación (botón B y luego A) para que el kernel desmonte correctamente, ya que si apaga jalando del cable que conecta a la corriente puede corromper los archivos.
- Se recomienda que las ROMs que se deseen cargar lo hagan por medio de la USB ya que ya hay un sistema incluido que evita hacerlo por otros medios y con riesgos de que se corrompan los datos o se pierdan, además de que los acomoda automáticamente dependiendo de la consola.

5. Descripción de los módulos de software

El subsistema de entrada y emulación está compuesto por dos módulos: `entrada.py`, que abstrae todos los dispositivos de control, y `emulador.py`, que gestiona el lanzamiento de mednafen y la pantalla de controles previa al juego.

Acción lógica	Botón gamepad	Tecla teclado	Descripción
ARRIBA	Hat ↑ / Eje Y < -0.5	↑	Navegar hacia arriba en el menú

ABAJO	Hat ↓ / Eje Y > +0.5	↓	Navegar hacia abajo en el menú
IZQUIERDA	Hat ← / Eje X < -0.5	←	Reservado para uso futuro
DERECHA	Hat → / Eje X > +0.5	→	Reservado para uso futuro
CONFIRMAR	Botón 0 (A)	Enter	Seleccionar juego / confirmar
ATRÁS	Botón 1 (B)	Escape	Cancelar / menú anterior
INICIO	Botón 7 (Start)	F1	Confirmar apagado del sistema

Tabla 2. Mapeo de acciones lógicas a controles físicos

5.1 entrada.py

La clase Entrada se puede ver como un traductor que nos ayuda a que se comuniquen el mando con el sistema, traduce cada uno de los botones de nuestro mando como lo es ARRIBA, ABAJO, ACEPTAR o ATRÁS. Con esto si se usa algún otro mando solo es necesario que se modifique este archivo y no todos los demás módulos. Al instanciar la clase se inicializa pygame.joystick y se detecta si hay algún mando conectado. Al detectarse uno, el sistema lo prepara y empieza a leer cada uno de los botones.

5.2 emulador.py

La clase Emulador se ocupa de todo lo que tiene que ver con el paso al emulador Mednafen, además de salirse del mismo y esto ayudado del módulo de **galería.py** para las pantallas que se ven antes de que inicie un juego.

Consola	Flag mednafen	Extensiones	Notas
NES	-force_module nes	.nes	Nintendo Entertainment System
SNES	-force_module snes	.smc / .sfc	Super Nintendo Entertainment System
GBA	-force_module gba	.gba	Game Boy Advance

Tabla 3. Consolas soportadas y configuración de mednafen

5.2.1 Pantalla de controles

Antes de entrar a mednafen, el método mostrar_controles() muestra la imagen que tiene la distribución de botones para cada consola. El nombre del juego aparece en la parte superior en color dorado. En la parte inferior parpadea el mensaje “Presiona A para jugar | Escape para volver” con un periodo de 0.5 segundos.

Método	Clase	Descripción
leer()	Entrada	Lee eventos pygame y devuelve una acción lógica

<code>_tecla_a_accion(tecla)</code>	Entrada	Mapea código de tecla a acción lógica
<code>_hat_a_accion(valor)</code>	Entrada	Convierte valor del D-pad a acción lógica
<code>_eje_a_accion(eje, valor)</code>	Entrada	Convierte eje analógico a acción (umbral 0.5)
<code>lanzar(consola, nombre, ruta)</code>	Emulador	Orquesta el ciclo completo de lanzamiento del juego
<code>_mostrar_controles(consola, nombre)</code>	Emulador	Muestra imagen de controles con texto parpadeante
<code>_mostrar_error(mensaje)</code>	Emulador	Despliega mensaje de error durante 3 segundos

Tabla 4. Métodos de los módulos `entrada.py` y `emulador.py`

6. Configuración de la Raspberry Pi

6.1 Instalación automática de la Retro Consola

Para la instalación solo es necesario descargar con el comando que se presenta en la parte de abajo para que comience una descarga de todo lo necesario:

```
$ sudo bash -c "$(curl -fsSL
https://raw.githubusercontent.com/nasarukey/sdk-retrogames/main/src/installer.sh)"
```

Nuestro instalador llamado `instalar.sh` comenzará a hacer los siguientes puntos:

- Instala dependencias.
- Descarga el proyecto.
- Crea carpetas de ROMs.
- Configura Mednafen.
- Da permisos al usuario.
- Configura USB y apagado.
- Crea el servicio de arranque automático.
- Oculta mensajes de inicio.
- Reinicia la Raspberry.

6.2 Instalación de mednafen

Mednafen se instala desde los repositorios oficiales de Raspberry Pi OS:

```
sudo apt update
sudo apt install -y mednafen
```

6.3 Configuración de mednafen para KMS/DRM

Mednafen utiliza SDL2 para su salida de video. Para que funcione correctamente en Raspberry Pi OS Lite sin escritorio, las variables de entorno SDL deben pasarse explícitamente al subprocesso:

```
entorno = os.environ.copy()
entorno['SDL_VIDEODRIVER'] = 'kmsdrm'
entorno['SDL_AUDIODRIVER'] = 'alsa'
proceso = subprocess.Popen(comando, env=entorno)
```

6.4 Archivo de configuración mednafen.cfg

mednafen genera un archivo de configuración en `~/mednafen/mednafen.cfg` la primera vez que se ejecuta. El sistema incluye una versión pre-configurada optimizada para la Raspberry Pi en `src/config/mednafen/mednafen.cfg`, que `main.py` copia al directorio correcto en cada arranque. Los parámetros más relevantes para este subsistema son:

- `video.driver: kmsdrm` — driver de video para pantalla sin escritorio
- `video.fs: 1` — forzar pantalla completa
- `sound.driver: alsa` — driver de audio para ALSA
- `input.joystick.axis_threshold: 0.5` — zona muerta de ejes analógicos

6.5 Estructura de carpetas de ROMs

Las ROMs deben colocarse en subcarpetas según la consola dentro de `src/roms/`:

```
src/  
  roms/  
    nes/      ← archivos .nes  
    snes/     ← archivos .smc o .sfc  
    gba/      ← archivos .gba
```

6.6 Permisos de usuario

El usuario que ejecuta la aplicación debe pertenecer a los grupos `input` (para gamepads), `audio` y `video`:

```
sudo usermod -aG video,audio,input,render $USER
```

Es necesario cerrar sesión y volver a iniciarla para que los cambios de grupo surtan efecto.

7. Desarrollo de los módulos

7.1 método leer()

`leer()` procesa la cola de eventos de `pygame` y devuelve la primera acción significativa que encuentre, o `ACCION_NINGUNA` si no hay eventos relevantes:

```
def leer(self) -> str:  
    pygame.event.pump()  
    for evento in pygame.event.get():  
        if evento.type == pygame.QUIT:  
            return ACCION_ATRAS  
        if evento.type == pygame.KEYDOWN:  
            return self._tecla_a_accion(evento.key)  
        if evento.type == pygame.JOYDEVICEADDED:  
            self.joystick =  
pygame.joystick.Joystick(evento.device_index)  
            self.joystick.init()
```

```

        if evento.type == pygame.JOYDEVICEREMOVED:
            self.joystick = None
        if evento.type == pygame.JOYBUTTONDOWN:
            return self.MAPA_BOTONES.get(evento.button, ACCION_NINGUNA)
        if evento.type == pygame.JOYHATMOTION:
            return self._hat_a_accion(evento.value)
        if evento.type == pygame.JOYAXISMOTION:
            return self._eje_a_accion(evento.axis, evento.value)
    return ACCION_NINGUNA

```

7.1.1 Conversión de ejes analógicos

El método `_eje_a_accion()` aplica el umbral de zona muerta y mapea el eje 0 (horizontal) y el eje 1 (vertical) a las acciones de navegación:

```

def _eje_a_accion(self, eje: int, valor: float) -> str:
    if abs(valor) < self.UMBRAL_EJE: # zona muerta 50%
        return ACCION_NINGUNA
    if eje == 0: # eje horizontal
        return ACCION_DERECHA if valor > 0 else ACCION_IZQUIERDA
    if eje == 1: # eje vertical
        return ACCION_ABAJO if valor > 0 else ACCION_ARRIBA
    return ACCION_NINGUNA

```

7.2 Clase Emulador — método lanzar()

El método `lanzar()` implementa el protocolo completo de sesión y recuperación del framebuffer descrito en la sección 3.5:

```

def lanzar(self, consola, nombre_rom, ruta_rom):
    if not self._mostrar_controles(consola, nombre_rom):
        return
    ejecutable = self.mapa_emuladores.get(consola, 'mednafen')
    flags = self.FLAGS_MEDNAFEN.get(consola, [])
    comando = [ejecutable] + flags + [ruta_rom]
    try:
        pygame.mixer.stop()
        pygame.display.quit() # liberar framebuffer
        time.sleep(1) # esperar liberación completa
        entorno = os.environ.copy()
        entorno['SDL_VIDEODRIVER'] = 'kmsdrm'
        entorno['SDL_AUDIODRIVER'] = 'alsa'
        self.proceso = subprocess.Popen(comando, env=entorno)
        self.proceso.wait() # bloquear hasta que termine
    except FileNotFoundError:
        print(f'Error: mednafen no encontrado')
    finally:
        self.proceso = None
        time.sleep(0.5) # esperar que mednafen libere
        pygame.display.init() # recuperar framebuffer
        pygame.mixer.init()
        self.pantalla.superficie = pygame.display.set_mode(
            (self.pantalla.ancho, self.pantalla.alto),
            pygame.FULLSCREEN
        )
        pygame.mouse.set_visible(False)

```

7.3 Pantalla de controles con texto parpadeante

El efecto de parpadeo del texto en la pantalla de controles se implementa sin temporizadores de pygame para evitar dependencias del estado del display durante la transición. Se usa `time.time()` para calcular el tiempo transcurrido desde el último cambio de estado:

```
parpadeo = True
ultimo_flip = 0
while True:
    ahora = time.time()
    # ... procesar eventos ...
    if ahora - ultimo_flip > 0.5:      # cambiar cada 500 ms
        parpadeo = not parpadeo
        ultimo_flip = ahora
        # ... redibujar pantalla ...
        if parpadeo:
            self.pantalla.dibujar_texto(
                '>> Presiona A para jugar | Escape para volver <<',
                ..., color=(50, 255, 50)
            )
        self.pantalla.actualizar()
    reloj.tick(30)
```

8. Integración del subsistema

Los módulos `entrada.py` y `emulador.py` se instancian en `main.py` y se pasan por referencia a la clase `Galeria`. La galería llama a `entrada.leer()` en cada iteración de su loop principal para obtener la acción del usuario. Cuando el usuario selecciona una ROM, la galería llama a `emulador.lanzar()` que bloquea hasta que el juego termina y luego devuelve el control al loop de la galería.

La referencia al objeto `Emulador` también se pasa al `MonitorUSB`, que la usa para terminar el proceso de mednafen (`self.proceso.terminate()`) si se inserta un USB con ROMs mientras se está jugando. Esto demuestra cómo el diseño basado en referencias a objetos permite la comunicación entre módulos sin acoplamiento directo.

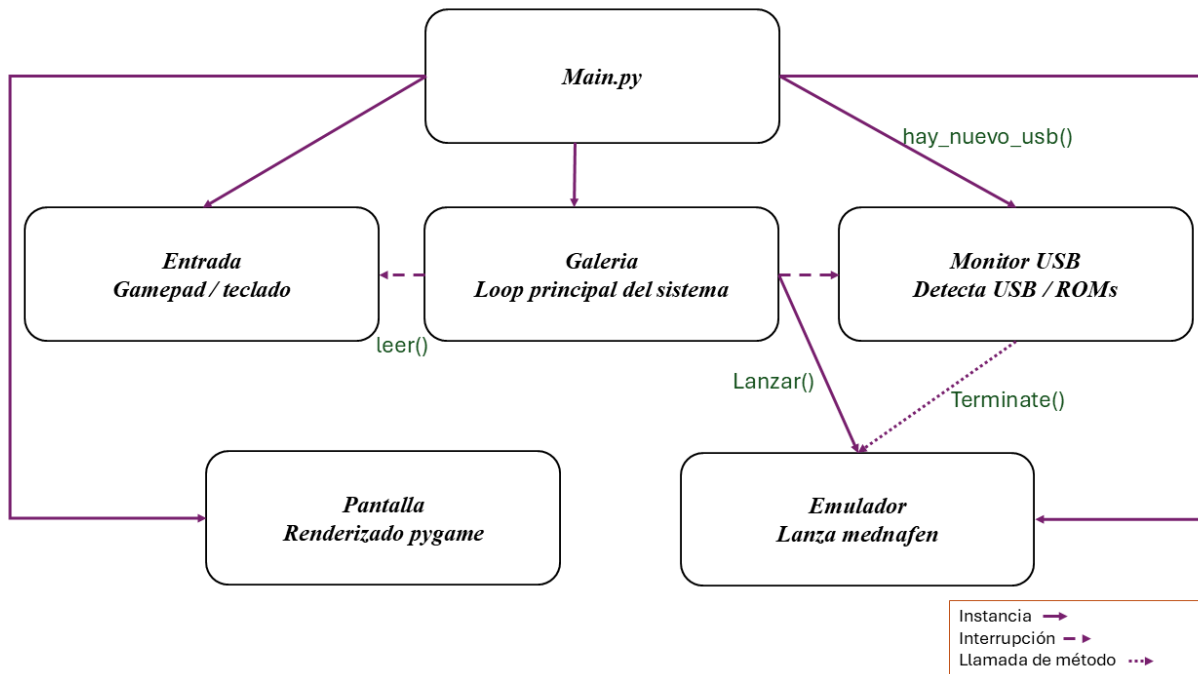


Figura 1. Interacción entre Galeria, Entrada, Emulador y MonitorUSB

9. Solución de problemas comunes

9.1 mednafen no inicia: 'No available video device'

Este error indica que pygame no liberó el framebuffer antes de que mednafen intentara inicializarlo. Verificar que `pygame.display.quit()` se llama antes de `subprocess.Popen()` y que se espera al menos 1 segundo entre ambas llamadas. También verifica que `SDL_VIDEODRIVER=kmsdrm` está en el entorno del subprocesso.

9.2 El control no es detectado al arrancar

Verificar que el usuario pertenece al grupo `input` con `groups $USER`. Si el control se conectó después del arranque, pygame debería detectarlo automáticamente mediante `JOYDEVICEADDED`. Si no es así, verificar que el driver del control está cargado con `lsusb` y `dmesg | tail`.

9.3 Deriva de ejes analógicos (el menú navega solo)

Si el menú navega sin que el usuario toque el control, el umbral de zona muerta puede ser insuficiente para el gamepad específico utilizado. Aumentar el valor de `UMBRAL_EJE` en la clase `Entrada` de 0.5 a 0.6 o 0.7 según sea necesario.

9.4 Pantalla en negro al regresar de mednafen

Si la pantalla queda en negro al regresar del emulador, el tiempo de espera después de que mednafen termina puede ser insuficiente. Aumentar el valor de `time.sleep(0.5)` en el bloque `finally` del método `lanzar()` a 1.0 segundo.

9.5 mednafen no reconoce el formato de la ROM

Verifica que la extensión del archivo es la esperada. Si mednafen devuelve un error de formato, el archivo ROM puede estar corrupto o en un formato de contenedor no soportado.

10. Evidencias

Repositorio del proyecto: https://github.com/nasaruke/SDK_RetroGames

Video demostrativo: <https://youtu.be/pv0NvIFaCqE>

11. Conclusiones

El proyecto nos ayudó a corroborar todos los aprendizajes en lecciones pasadas de las practicas, sobre todo de la de kiosco multimedia ya que esa fue un inicio de como podíamos esconder las llamadas y el arranque de la raspberry que fue donde creo este proyecto nos atoro más, puesto que por más que intentábamos ocultarlas no nos fue posible hacerlo en un 100%. Otras dificultades por las que atravesamos fueron en el arranque y salida de Mednafen pues llego un punto del proyecto en el que al intentar correr un juego desde el menú no había ningún problema, pero, cuando se necesitaba que reingresara al menú nos mandaba directo a la línea de comando.

Con esto puedo decir que se cumplió el poder crear un sistema embebido desde cero y que encontré muy asombroso ya que, con hardware muy accesible y robusto, con la integración de una interfaz para el usuario con una gestión automática de USBs y ROMs que facilita la carga de videojuegos y que esto no obstruye con una experiencia fluida dentro de los juegos.

En cuanto a la emulación se puede aparentar que solo con la descarga del Mednafen o cualquier otro emulador y escribir el comando para arrancar la rom es todo, sin embargo, el crear un entorno que nos ayude a facilitar esto para que no sea necesario el escribir o configurar cada juego que juguemos, sino, mas bien automatizar todos estos pasos para ofrecer una mejor experiencia de juego.

12. Cuestionario

1. ¿Por qué se utiliza doble buffer (display.flip) en lugar de actualizar la pantalla píxel a píxel?
2. ¿Qué ventaja tiene encapsular todas las llamadas a pygame dentro de la clase Pantalla en lugar de llamar a pygame directamente desde cada módulo?
3. ¿Por qué se define un umbral de 0.5 para los ejes analógicos en lugar de detectar cualquier valor distinto de cero? ¿Qué problema resuelve esto en la práctica?
4. Describe el problema del framebuffer compartido entre pygame y mednafen. ¿Por qué es necesario llamar a pygame.display.quit() antes de lanzar mednafen y cómo se recupera la pantalla al regresar?
5. ¿Qué función cumple el threading.Lock en la clase MonitorUSB? ¿Qué condición de carrera podría ocurrir si se eliminara el lock?
6. ¿Por qué se utiliza pyudev para detectar la inserción de USB en lugar de revisar periódicamente el contenido de /dev? Describe las ventajas del enfoque basado en eventos sobre el polling.

13. Bibliografía

[1] E. A. Lee y S. A. Seshia, Introduction to Embedded Systems: A Cyber-Physical Systems Approach, 2.a ed. Cambridge, MA: MIT Press, 2017.

[2] SDL Community, "SDL2 Wiki — Joystick," 2024. [En línea]. Disponible en: <https://wiki.libsdl.org/SDL2/CategoryJoystick>

[3] Mednafen Team, "Mednafen Documentation," 2024. [En línea]. Disponible en: <https://mednafen.github.io/documentation/>

[4] D. J. Greenwalt, "Linux KMS/DRM Architecture," Linux Kernel Documentation, 2023. [En línea]. Disponible en: <https://www.kernel.org/doc/html/latest/gpu/drm-kms.html>

[5] Python Software Foundation, "subprocess — Subprocess management," Python 3.11 Documentation, 2024. [En línea]. Disponible en: <https://docs.python.org/3/library/subprocess.html>
